

```
int iamprivate;
void privateMethod()
{
    System.out.println("privateMethod");
}
}
```

If you use multiple objects from the same package, they can access each other's package-access methods and data variables directly, merely by referencing an object that has been instantiated. With package access, other objects in the package don't have to bother going through the methods. They can get right to the variables. So, when you don't include any access modifier, you are in fact giving your variable package access.

```
int x; // package access instance variable
Notice, it's declared neither public nor private.
```

5.2.3 Protected

Protected allows the class itself, Subclasses and all classes in the same package to access the members. Generally speaking, protected offers greater access than package access. Use the protected access level when it's appropriate for a class's Subclasses to have access to the member, but not for unrelated classes to have access. Protected members are like family secrets—you don't mind if someone in the family knows—but you don't want outsiders to know.

The Access Modifier protected can be used on either a method or an encapsulated data variable. Protected serves as a middle ground between the private and public access modifier. A Superclass's protected data variables may be accessed only by: methods of the Superclass, methods of the Subclass or methods of other classes in the same package.

To summarise protected members have package access. Protected access means the variable is public to the class, private outside the class, and public to the subclasses. It's like the Subclasses can reach up and get access to the data variables of the Superclass.

5.3 Super reference

Already, we know that the Superclass-Subclass interaction has a big effect on the Constructor. The Superclass Constructors are either implicitly or explicitly called first in the Constructor of a Subclass.

```
super( whatever the Superclass needs)
```